"# Python Libraries

1. What is a Library?

A **Library** is a collection of ready-made, reusable code that we can use in our own Python programs.

Instead of writing everything from the beginning, we can use code that other developers have already created.

Example

```python
import math

print(math.sqrt(25))
```

Output:

```text
5.0
```

Here:

```text
math        Library / module
sqrt()      Function
25          Input
5.0         Result
```

Simple Definition

> A library is reusable code that helps us perform tasks without writing everything ourselves.

---

2. Why Do We Use Libraries?

Libraries make programming easier and faster.

Main Benefits

1. Code Reuse

We can use existing code instead of writing it again.

```python
import math

print(math.sqrt(100))
```

We don't need to create our own square-root function.

---

2. Faster Development

Libraries provide ready-made functions and tools.

For example:

```python
import random

print(random.randint(1, 10))
```

We can generate a random number without implementing the random-number algorithm ourselves.

---

3. Less Code

Without libraries, we may need to write many lines of code.

With libraries, we can often do the same task with a few lines.

---

4. Tested Functionality

Popular libraries are usually used and tested by many developers.

This can save development and testing time, although we should still understand and verify the library we use.

---

5. Easy Maintenance

Using well-designed libraries can make programs easier to maintain.

For example, instead of writing complex networking code ourselves, we can use a library such as `requests`.

---

3. Module vs Package vs Library

These three terms are related, but they are not exactly the same.

Module

A **Module** is usually a single Python file.

Example:

```text
math_tools.py
```

It can contain:

```python
def add(a, b):
return a + b
```

We can import it:

```python
import math_tools

print(math_tools.add(10, 20))
```

Easy Definition

> **Module = Usually one `.py` file.**

---

4. What is a Package?

A **Package** is a way to organize related Python modules together.

For example:

```text
myproject/
tools/
__init__.py
math_tools.py
string_tools.py
```

Here:

```text
tools
Package
math_tools.py
Module
string_tools.py
Module
```

Easy Definition

> **Package = A collection/organization of related Python modules.**

Modern Python also supports **namespace packages**, so an `__init__.py` file is not always required.

---

5. What is a Library?

A **Library** is a broader term for reusable code that developers can use in their programs.

A library may contain:

```text
Modules
Packages
Functions
Classes
Tools
```

Examples include:

```text
NumPy
Pandas
Requests
Django
```

Easy Definition

> **Library = Reusable code that provides useful functionality.**

---

6. Module vs Package vs Library

Term	Simple Meaning	Example
Module	Usually one Python file	`math_tools.py`

| Package | Organized collection of modules | `tools/` |

| Library | Reusable code for solving problems | NumPy, Pandas |

Easy Memory Trick

```text
Module

One Python file

Package

Group of related modules

Library

Reusable code for a particular purpose
```

---

7. Standard Library

Python comes with a large collection of built-in modules called the **Python Standard Library**.

You usually do not need to install these separately.

Examples

```text
math

random

datetime

os
```

sys

json

re

```

---

## `math`

Used for mathematical operations.

```python

import math

print(math.sqrt(25))

print(math.pi)

```

Output:

```text

5.0

3.141592653589793

```

`random`

Used to generate random values.

```python

import random

print(random.randint(1, 10))

```

Possible output:

```text

7

```

The result can be different each time.

`datetime`

Used to work with dates and times.

```python

from datetime import datetime

now = datetime.now()

print(now)

```

`os`

Used for operating-system-related tasks.

```python

import os

print(os.getcwd())

```

This prints the current working directory.

---

sys

Used to interact with the Python interpreter and system-related information.

Example:

```python
import sys

print(sys.version)
```

It can also be used for command line arguments:

```python
import sys

print(sys.argv)
```

---

json

Used to work with JSON data.

```python
import json

data = '{"name": "Faruk", "age": 25}'
```
```

result = json.loads(data)

print(result["name"])
```

Output:

```text
Faruk
```

`re`

Used for **Regular Expressions**.

```python
import re

text = "My number is 12345"

result = re.findall(r"\d+", text)

print(result)
```

Output:

```text
['12345']
```

```

8. Third-Party Libraries

Third-party libraries are libraries created outside the Python standard library.

They usually need to be installed separately.

For example:

```bash
pip install requests
```

Then we can use:

```python
import requests
```

---

9. Common Third-Party Python Libraries

Some popular Python libraries and frameworks are:

```text
NumPy
Pandas
Requests
Django
Flask
FastAPI
Scrapy
```

BeautifulSoup

Selenium

PyTorch

TensorFlow

```

---

## NumPy

**NumPy** is mainly used for:

```text

Numerical computing

Arrays

Mathematical operations

Scientific computing

```

Example:

```python

import numpy as np

numbers = np.array([1, 2, 3, 4])

print(numbers)

```

---

10. Pandas
```

Pandas is mainly used for:

```text
Data analysis
Data manipulation
Tables
CSV files
DataFrames
```

Example:

```python
import pandas as pd

data = {
"Name": ["Alice", "Bob"],
"Age": [20, 25]
}

df = pd.DataFrame(data)

print(df)
```

---

11. Requests

Requests is used to send HTTP requests.

For example:

```python
import requests

response = requests.get("https://example.com")

print(response.status_code)
```

It is commonly used when working with:

```text
APIs
Web requests
HTTP
Web data
```

---

12. Django

Django is a Python web framework.

It is used to build:

```text
Web Applications
Websites
Backend Systems
REST APIs
```

Example:

```text
Django

Python Web Framework

Web Application
```

---

13. Flask

Flask is a lightweight Python web framework.

It can be used to create:

```text
Web Applications

REST APIs

Backend Services
```

Simple example:

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")

def home():

return "Hello World"
```

---

14. FastAPI

FastAPI is a modern Python framework mainly used for building APIs.

It is popular for:

```text
REST APIs
Backend Services
API Development
```

Example:

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
return {"message": "Hello World"}
```

---

15. Scrapy

Scrapy is a Python framework used for web crawling and web scraping.

It can collect data from websites.

Common uses:

```text
Web Scraping
Web Crawling
Data Extraction
```

---

16. BeautifulSoup

BeautifulSoup is used to parse HTML and XML documents.

It is commonly used for extracting data from web pages.

Example:

```python
from bs4 import BeautifulSoup

html = "Hello "

soup = BeautifulSoup(html, "html.parser")

print(soup.h1.text)
```

Output:

```text
Hello
```

---

17. Selenium

Selenium is used to automate web browsers.

It can perform actions such as:

```text
Open browser
Click buttons
Fill forms
Navigate pages
Extract information
Test websites
```

It is commonly used for browser automation and testing.

---

18. PyTorch

PyTorch is a machine learning and deep learning framework.

It is commonly used for:

```text
Machine Learning
Deep Learning
Neural Networks
Computer Vision
Natural Language Processing
```

```

---

# 19. TensorFlow

**TensorFlow** is another popular machine learning and deep learning framework.

It can be used for:

```text
Machine Learning
Deep Learning
Neural Networks
AI Applications
```

---

20. Standard Library vs Third-Party Library

Standard Library	Third-Party Library
Comes with Python	Usually installed separately
Usually no `pip install` needed	Often installed using `pip`
`math`	`numpy`
`random`	`pandas`
`datetime`	`requests`
`os`	`django`
`sys`	`flask`
`json`	`fastapi`
```

| `re` | `scrapy` |

Example

Standard library:

```python
import math
```

No separate installation is normally required.

Third-party:

```bash
pip install numpy
```

Then:

```python
import numpy
```

---

21. How to Install a Third-Party Library

We commonly use `pip`.

General Syntax

```bash
pip install library_name
```

Example:

```bash
pip install requests
```

Then:

```python
import requests
```

Another example:

```bash
pip install numpy
```

Then:

```python
import numpy
```

---

22. Importing a Library

There are several common ways to import Python code.

Import the whole module

```python
```

```
import math

print(math.sqrt(25))
```

Import with an alias

```python
import numpy as np

numbers = np.array([1, 2, 3])
```

Here:

```text
numpy  actual module name
np     short alias
```

Import a specific function

```python
from math import sqrt

print(sqrt(25))
```

Here we do not need:

```python
```

math.sqrt()

```

We can directly use:

```python

sqrt()

```

Import Multiple Items

```python

from math import sqrt, pi

print(sqrt(25))

print(pi)

```

---

23. Why Are Libraries Important in Python?

Python has a very large ecosystem of libraries.

Different libraries help with different tasks.

```text

Python Libraries

Web Data AI/ML

Django Pandas PyTorch

Flask NumPy TensorFlow

FastAPI

APIs / Backend

```

Other areas:

```text
Web Scraping

Scrapy

BeautifulSoup

Selenium

Data Science

NumPy

Pandas

Machine Learning

PyTorch

TensorFlow

Web Development

Django

Flask

FastAPI
```

```

24. Important Interview Questions

Q1. What is a Python Library?

A Python Library is reusable code that provides functions, classes, or tools for solving common problems.

Q2. Why do we use libraries?

Main reasons:

```text
Code reuse
Faster development
Less code
Useful ready-made functionality
Easier maintenance
```

Q3. What is a Module?

A module is usually a single Python file containing Python code.

Example:

```text
math_tools.py
```

Q4. What is a Package?

A package is a way to organize related Python modules together.

Q5. What is the Python Standard Library?

It is the collection of modules that come with Python and can usually be used without installing separate packages.

Examples:

```text
math
os
sys
json
re
datetime
```

Q6. What is a Third-Party Library?

A third-party library is Python code developed outside the Python standard library.

It is often installed using:

```bash
pip install package_name
```

Q7. What is `pip`?

`pip` is the standard package installer commonly used to install Python packages from the Python Package Index (PyPI).

Example:

```bash
pip install requests
```

Q8. What is the difference between `import` and `from ... import`?

`import`

```python
import math

print(math.sqrt(25))
```

We access the function through the module.

`from ... import`

```python
from math import sqrt

print(sqrt(25))
```

We import the specific item directly.

---

25. Quick Revision

Library

```text
Reusable code
```

Module

```text
Usually one .py file
```

Package

```text
Collection/organization of related modules
```

Standard Library

```text
Comes with Python
```

Examples:

```text
math
```

os

sys

json

re

datetime

```

### Third-Party Library

```text

Usually installed separately

```

Examples:

```text

NumPy

Pandas

Requests

Django

Flask

FastAPI

Scrapy

BeautifulSoup

Selenium

PyTorch

TensorFlow

```

Install
```

```bash
pip install package_name
```

Import

```python
import package_name
```

---

26. Final Summary

The main idea is:

```text
Python Library

Reusable Code

Save Development Time

Write Less Code

Build Programs Faster
```

Easy Mental Model

```text
Module

Usually one .py file
```

Package

Organizes related modules

Library

Reusable functionality

```

### Most Important Examples

```python
# Standard Library
import math

print(math.sqrt(25))
```

```python
# Third-Party Library
import requests

response = requests.get("https://example.com")
print(response.status_code)
```

One-Line Definition

> **A Python library is reusable code that helps us perform tasks without writing everything from scratch.**
```